

# 第九次上机作业：数值积分方法比较

学号：241830137 姓名：朱子健

2026 年 5 月 6 日

## 1 问题描述

数学上可以证明

$$\int_0^1 \frac{4}{1+x^2} dx = \pi. \quad (1)$$

本题分别使用复合梯形公式和复合 Simpson 公式计算  $\pi$  的近似值。选择不同的  $h$  (或等分数  $n$ )，对每种求积公式，试将误差刻画成  $h$  的函数，并比较两种方法的精度。特别关注是否存在某个  $h$ ，当低于这个值之后再继续减小  $h$ ，计算不再有所改进，并解释其原因。

## 2 算法原理

### 2.1 复合梯形公式

将积分区间  $[a, b]$  作  $n$  等分，步长  $h = (b - a)/n$ ，节点  $x_i = a + ih$ ， $i = 0, 1, \dots, n$ 。复合梯形求积公式为

$$T_n(f) = \frac{h}{2} \left[ f(a) + f(b) + 2 \sum_{i=1}^{n-1} f(a + ih) \right]. \quad (2)$$

其离散误差为  $O(h^2)$ 。

---

#### 算法 1 复合梯形公式

---

输入：区间端点  $a, b$ ，等分数  $n$ ，被积函数  $f(x)$

输出：积分近似值  $T$

```
1:  $h \leftarrow (b - a)/n$ 
2:  $s \leftarrow 0.5 \cdot f(a) + 0.5 \cdot f(b)$ 
3: for  $i \leftarrow 1$  to  $n - 1$  do
4:    $x \leftarrow a + i \cdot h$ 
5:    $s \leftarrow s + f(x)$ 
6: end for
7:  $T \leftarrow h \cdot s$ 
8: return  $T$ 
```

---

## 2.2 复合 Simpson 公式

将区间  $[a, b]$  作  $n$  等分, 其中  $n = 2m$  为偶数, 步长  $h = (b - a)/n$ 。在每两个子区间上应用 Simpson 公式, 得到复合 Simpson 求积公式

$$S_m(f) = \frac{h}{3} \left[ f(a) + f(b) + 4 \sum_{i=1}^m f(a + (2i-1)h) + 2 \sum_{i=1}^{m-1} f(a + 2ih) \right]. \quad (3)$$

其离散误差为  $O(h^4)$ 。

---

### 算法 2 复合 Simpson 公式

---

输入: 区间端点  $a, b$ , 等分数  $n$  ( $n$  为偶数), 被积函数  $f(x)$

输出: 积分近似值  $S$

```
1: if  $n$  为奇数 then
2:    $n \leftarrow n + 1$ 
3: end if
4:  $h \leftarrow (b - a)/n$ 
5:  $s \leftarrow f(a) + f(b)$ 
6: for  $i \leftarrow 1$  to  $n - 1$  do
7:    $x \leftarrow a + i \cdot h$ 
8:   if  $i$  为奇数 then
9:      $s \leftarrow s + 4 \cdot f(x)$ 
10:  else
11:     $s \leftarrow s + 2 \cdot f(x)$ 
12:  end if
13: end for
14:  $S \leftarrow h \cdot s/3$ 
15: return  $S$ 
```

---

## 2.3 误差停止改进的原因

随着  $h \rightarrow 0$ , 离散误差按  $O(h^2)$  或  $O(h^4)$  减小, 但浮点舍入误差随节点数增加而累积。当  $h$  极小 ( $n$  极大) 时, 舍入误差主导总误差, 导致误差不再减小甚至回升。在单精度 (float32) 下这一现象比双精度出现得更早且更明显。

## 3 结果分析

为了清晰地显示舍入误差的影响, 程序采用单精度浮点数 (`np.float32`) 进行计算, 并将  $n$  取至非常大。精确值取为  $\pi$  的双精度值。

从表 1 可以看出:

- 在  $n$  较小 ( $h$  较大) 时, 梯形公式误差约为  $O(h^2)$ , Simpson 公式误差约为  $O(h^4)$ , 后者精度明显更高。
- 当  $n$  增大到一定值后, 两种公式的误差均停止稳步下降, 出现波动甚至回升。梯形公式在  $n = 32$  左右误差最小, Simpson 公式在  $n = 32 \sim 128$  左右误差最小。

表 1: 复合梯形和复合 Simpson 公式在 float32 下的误差变化

| $n$   | $h$      | 梯形误差     | 改进? | Simpson 误差 | 改进? |
|-------|----------|----------|-----|------------|-----|
| 2     | 5.00e-01 | 2.14e-02 | —   | 1.14e-03   | —   |
| 4     | 2.50e-01 | 5.35e-03 | 是   | 7.41e-05   | 是   |
| 8     | 1.25e-01 | 1.33e-03 | 是   | 4.64e-06   | 是   |
| 16    | 6.25e-02 | 3.31e-04 | 是   | 1.91e-06   | 是   |
| 32    | 3.13e-02 | 8.47e-05 | 是   | 5.96e-07   | 是   |
| 64    | 1.56e-02 | 2.87e-05 | 是   | 1.79e-06   | 否   |
| 128   | 7.81e-03 | 1.70e-05 | 是   | 9.54e-07   | 是   |
| 256   | 3.91e-03 | 2.54e-05 | 否   | 4.17e-06   | 否   |
| 512   | 1.95e-03 | 1.14e-04 | 否   | 4.17e-06   | 否   |
| 1024  | 9.77e-04 | 9.01e-05 | 是   | 4.17e-06   | 否   |
| 2048  | 4.88e-04 | 6.13e-04 | 否   | 4.77e-06   | 否   |
| 4096  | 2.44e-04 | 4.96e-04 | 是   | 2.41e-05   | 否   |
| 8192  | 1.22e-04 | 1.83e-03 | 否   | 5.96e-05   | 否   |
| 16384 | 6.10e-05 | 4.97e-04 | 是   | 5.72e-05   | 是   |
| 32768 | 3.05e-05 | 6.65e-03 | 否   | 3.25e-04   | 否   |
| 65536 | 1.53e-05 | 3.33e-03 | 是   | 1.16e-03   | 否   |

- 此后继续增大  $n$  (减小  $h$ )，舍入误差逐渐主导，数值结果不再改进。由于使用 float32 单精度，表示精度有限（机器精度约  $1.2 \times 10^{-7}$ ），大量函数值相加引入的舍入噪声完全淹没了离散误差的减小。

## 4 结论

- 复合 Simpson 公式在同等步长下具有远优于复合梯形公式的精度，其误差以  $O(h^4)$  收敛，而梯形公式仅为  $O(h^2)$ 。
- 无论是复合梯形还是复合 Simpson 公式，都存在一个最优的  $h$  (或  $n$ )。当步长小于该值时，由于计算机浮点运算的舍入误差累积，近似值的总误差不再下降，甚至增大。
- 在 float32 单精度下，该现象非常显著；若改用 double 双精度，最优  $h$  可以小得多，但最终也会受到同样的限制。

## 5 附录：程序代码

### 5.1 复合梯形与复合 Simpson 公式 (float32)

```

1 import numpy as np
2 def f32(x):
3     """单精度被积函数"""
4     return np.float32(4.0) / (np.float32(1.0) + np.float32(x)**2)
5

```

```

6 def composite_trapezoidal_float32(a, b, n):
7     """复合梯形公式, float32单精度"""
8     h = np.float32((b - a) / n)
9     s = np.float32(0.5) * f32(a) + np.float32(0.5) * f32(b)
10    for i in range(1, n):
11        s += f32(a + np.float32(i) * h)
12    return float(h * s)
13
14 def composite_simpson_float32(a, b, n):
15     """复合Simpson公式, float32单精度"""
16     if n % 2 != 0:
17         n += 1
18     h = np.float32((b - a) / n)
19     s = f32(a) + f32(b)
20     for i in range(1, n, 2):
21         s += np.float32(4.0) * f32(a + np.float32(i) * h)
22     for i in range(2, n, 2):
23         s += np.float32(2.0) * f32(a + np.float32(i) * h)
24     return float(h / np.float32(3.0) * s)
25
26
27 a, b = 0, 1
28 exact = np.pi
29
30 print("使用float32单精度, 误差不再减小的现象更显著: \n")
31 print(f"{'n':<10}_{'h':<14}_{'梯形误差':<16}_{'有改进?':<10}_{'Simpson误差':<16}_{'有改进?':<10}")
32 print("-" * 76)
33
34 prev_T = None
35 prev_S = None
36
37 for n in [2, 4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048, 4096, 8192, 16384, 32768,
38          65536]:
39     h = (b - a) / n
40     err_T = abs(composite_trapezoidal_float32(a, b, n) - exact)
41     err_S = abs(composite_simpson_float32(a, b, n) - exact)
42
43     if prev_T is None:
44         T_better, S_better = "-", "-"
45     else:
46         T_better = "是" if err_T < prev_T else "否"
47         S_better = "是" if err_S < prev_S else "否"
48
49     print(f"{'n':<10}_{'h':<14.6e}_{'err_T':<16.6e}_{'T_better':<10}_{'err_S':<16.6e}_{'S_better':<10}")
50
51     prev_T = err_T
52     prev_S = err_S

```